RT-FedMTL: A Real-Time Federated Multi-Task Learning Framework with Small Language Models for Natural Language Understanding Tasks

Quang-Vinh Pham
Department of Data Science
TMA Tech Group, Gia Lai, Vietnam
Email: pqvinh@tma.com.vn

Quang-Hung Le
Department of Information Technology
Quy Nhon University, Vietnam
Email: lequanghung@qnu.edu.vn

***Abstract*—The adoption of Federated Learning (FL) for privacy-preserving Natural Language Understanding (NLU) is often hindered by the resource constraints of edge devices and the complexity of Multi-Task Learning (MTL) environments. This paper proposes a Real-Time Federated Multi-Task Learning framework using Small Language Models (SLMs), termed RT-FedMTL. We systematically evaluate five SLM architectures across benchmarks from the General Language Understanding Evaluation (GLUE) suite under multiple paradigm-complexity stages and data distributions. Our findings reveal that RT-FedMTL achieves a per-client resource reduction of 74–93% relative to centralized training, though it incurs a 2.2–3.7× training time overhead due to real-time synchronization. We identify a critical architectural trade-off: while DistilBERT offers the highest robustness for sentiment analysis, MiniLM achieves an optimal balance, matching or exceeding larger models in MTL benchmarks like QQP and STS-B with a 3× smaller memory footprint. We further demonstrate that Low-Rank Adaptation (LoRA) effectively mitigates communication bottlenecks in this real-time setting. The source code is available at: https://github.com/vinh1988/RT-FedMTL.git.**

***Index Terms*—Federated Learning, Small Language Models, Multi-Task Learning, Data Heterogeneity, Privacy-Preserving Machine Learning**

I. INTRODUCTION

With the proliferation of NLU systems on mobile devices, enterprise platforms, and edge services, the demand for privacy-preserving collaborative learning has grown significantly [1]. Centralized collection of NLU data from sources such as personal messages, medical text, or behavioral logs is often restricted by strict privacy regulations and user concerns. Consequently, FL has emerged as a key solution by keeping raw data local and sharing only model updates [2]. Modern NLU applications, ranging from sentiment analysis for customer feedback to real-time intent classification for conversational agents and semantic similarity for information retrieval, increasingly require the ability to learn from diverse, distributed data streams while supporting MTL simultaneously [3], [6]. The trend makes centralized single-pipeline training less feasible and encourages FL training models, which can learn collaboratively without breaking the data locality.

While FL is a promising technique for NLU, there are three major concerns, as highlighted by benchmarking frameworks like FedNLP [4]. First, client data usually is non-IID. Given that diverse users and organizations write language data with different vocabularies, label compositions and preferences, convergence is typically slowed, and global model quality is poor [5], [3]. Second, task heterogeneity is common in realistic deployments. A federation could be made with intent classification, sentiment analysis, question answering, or even generation-related goals all at once [3], [6], [7]. Classical one-model FL methods are not suited to this mixed-task regime. Third, communication and scalability are essential. In general, the repeated syncing of multiple clients can lead to significant overhead, particularly when the size of the model is large and the connected clients unstable [2], [6]. Recent literature on federated MTL (e.g., MOCHA [3], FedBone [6], MIRA [8], and encoder–decoder based M-Fed [7]) indicates that data heterogeneity and task heterogeneity need to be considered as first-class design considerations.

SLMs are especially appealing in federated systems because they improve efficiency without losing language pretraining functionality. Key advantages include:

- Communication efficiency: A smaller parameter set reduces the communication cost per round (both upload and download).
- Reduced computation cost: SLMs use fewer resources of local memory and training time, allowing for broader client involvement.

- Edge-device deployment: compact models are easier to deploy and update on resource-constrained devices.

This role translates into parameter-efficient adaptation techniques (e.g., adapter-style and low-rank updates) that preserve competitive performance while decreasing federated overhead.

Current FL pipelines predominantly rely on batch-based synchronization, where clients execute multiple local steps and synchronize periodically [2], [5]. While robust for static datasets, this design is less suited for streaming and dynamic environments, where language patterns, subjects, and user intent evolve over time. To address these limitations, this paper proposes RT-FedMTL, a real-time FL-MTL framework specifically designed for low-latency synchronization and agile adaptation. By utilizing WebSocket protocols and optimizing for SLMs, RT-FedMTL reduces the feedback loop, enabling near-instantaneous global updates that are essential for responsive applications like chatbots, live moderation, and event-driven analytics.

This work makes several key contributions to the field of federated NLU:

- We introduce RT-FedMTL, a novel real-time federated framework that integrates MTL and SLMs to leverage cross-task knowledge transfer while minimizing on-device resource footprints.
- We evaluate five SLM architectures (DistilBERT [13], BERT-Medium [20], BERT-Mini [20], MiniLM [14], and TinyBERT [30]) across three benchmark NLU tasks (sentiment analysis, paraphrase detection, and semantic similarity) to identify the optimal balance between performance and resource consumption.
- We analyze the impact of non-IID and task heterogeneity on convergence speed and task accuracy in a real-time environment.
- We demonstrate that RT-FedMTL achieves a significant reduction in per-client GPU memory usage (up to 74–93%) compared to centralized MTL baselines while maintaining high performance across multiple benchmarks from the GLUE suite.

The rest of the paper is structured as follows. Section II reviews the relevant work in FL and MTL. Section III defines the formal problem of RT-FedMTL. In Section IV we detail the system architecture and optimization strategies. The experimental setup and results analysis are described in Section V. Finally, Section VI wraps up the paper and presents the next steps.

II. RELATED WORK

This section reviews the foundational research and current advancements in FL for NLP, MTL, and the development of SLMs. We evaluate how these fields intersect to enable efficient distributed learning and identify the research gaps that our proposed real-time framework aims to address.

A. FL for NLP

There are many examples of FL utilized in NLP/NLU applications such as text classification and personalized language services where raw text cannot be aggregated [2], [5]. Basic optimization methods like FedAvg [2] had set up the typical client-update-server-aggregation loop, and FedProx [5] added regularization to enable better behavior against client drift. Advanced methods like SCAFFOLD [9] further address gradient heterogeneity to ensure stable convergence. Federated MTL approaches, such as MOCHA, demonstrate that inter-task relatedness can be made more powerful under heterogeneous setups by improving statistical efficiency and robustness to system issues [3]. Newer approaches such as FedBone [6], MIRA [8], and M-Fed [7] take this further by explicitly separating shared from task-dependent elements or using specialized encoder–decoder architectures to handle large-scale heterogeneous MTL environments. Notwithstanding this progress, a large number of FL-for-NLP studies rely on limited task diversity or mainly rely solely on single-task user personalization.

B. MTL in NLP

MTL in NLP generally employs one of the two paradigms: hard parameter sharing and soft parameter sharing [3], [10]. Hard sharing employs a common encoder, instead of a separate task head for each instance of the model, saving the model from overfitting and training cost. Soft sharing retains task-specific factors but promotes similarity either by regularization or by limiting interaction. The adoption of parameter-efficient fine-tuning (PEFT), such as adapters [11] and modular shared frameworks like AdapterHub [12], has further enabled efficient multi-task adaptation with minimal overhead. MTL improves representation learning by transfer of useful linguistic features between tasks. On the other hand, it also comes with known drawbacks, such as task imbalance (over-representation of dominant tasks with lower-level tasks) and negative transfer (one task harms others). These problems are more pronounced in the context of federated training because client involvement is partial, and the balance of data distribution between rounds is not consistent.

C. SLMs for Efficient NLP

SLMs like DistilBERT [13], MiniLM [14], and ALBERT [15] have been shown that careful compression and distillation can preserve good performance with a cost between parameter count and inference. The common reduction methods are knowledge distillation,

parameter sharing, low-rank factorization, and architectural simplification. These capabilities make those SLMs very appropriate for edge and distributed learning: they decrease the transmission overhead, reduce on-device training cost, and achieve faster federated iteration cycles. Recent innovations like EdgeDistilBERT [16] further push the boundaries of lightweight multi-task FL on edge devices. In settings with multi-task federated solutions, SLMs also facilitate scaling onto larger numbers of clients and tasks on fixed resource budgets for the system.

D. FL under Data Heterogeneity

Non-IID data is still one of the major bottlenecks in FL. Client-dependent language patterns, domain shifts, and label bias may lead to biased updates at the local level and may disrupt global optimization at the global level. This problem is aggravated in heterogeneous NLU deployments where the difference between client tasks exists (cross-client task diversity) whereby the clients can perform partial representations for optimizing different objectives. Strategies such as FedBN [17] and low-rank parameterizations like FedPara [18] have been investigated to mitigate these heterogeneity effects. As previous work report, aggregation without attention to the structure of a task will degrade velocity and final generalization, whereas when heterogeneity is mixed up (thereby aggregation). Since convergence speed can harm fine generalization and final speed depending on task structure, techniques such as personalized layers, proximal regularization, gradient conflict reducing, and encoder–decoder decomposition have been proposed to bridge this gap [5], [10], [7].

E. Research Gap

Existing studies provide strong building blocks but still leave an important gap for practical NLU systems, as identified in recent systematic reviews [19]:

- Lack of real-time FL frameworks: most methods are designed for periodic offline rounds rather than continuous adaptation.
- Limited integration of FL + MTL + SLM: many works treat efficiency, heterogeneity, and multi-task transfer separately.
- Insufficient heterogeneity analysis in NLU FL: current evaluations often miss how task diversity, data skew, and communication constraints interact.

This gap motivates a unified, real-time federated multi-task approach that combines efficient language modeling with heterogeneity-aware optimization for distributed NLU.

III. PROBLEM DEFINITION

This section formalizes the RT-FedMTL environment for NLU. We define the mathematical framework for distributed multi-task optimization, address the challenges of data and system heterogeneity, and establish the global objective for collaborative model training across private edge clients.

A. Formalization of RT-FedMTL for NLU

We consider a RT-FedMTL environment based on NLU in which training proceeds over communication rounds $t = 1, 2, \ldots$ [2], [3], [6]. During each round, a subset of clients is selected, local updates are performed on private data streams, and parameters are pushed to a central server for aggregation. Unlike offline FL, client data distributions may vary over time; therefore, both local goals and optimum parameters are time-dependent. Let $\mathcal{C} = \{1, \ldots, C\}$ be the set of clients and $\mathcal{K} = \{1, \ldots, K\}$ be the set of NLU tasks. Each client $c \in \mathcal{C}$ is associated with one primary task $\tau(c) \in \mathcal{K}$ (or a small task subset), where new samples are continuously added from a local stream. The objective is to learn a federated model that maintains cross-task transfer quality while respecting task-level specialization under non-IID and time-varying data.

B. Data and Task Heterogeneity in Clients

Heterogeneity comes in two forms:

- Heterogeneity of data: the local distributions in clients c_i, c_j will be different (i.e. $P_{c_i}(x, y) \neq P_{c_j}(x, y)$ (label skew, domain shift, vocabulary differences).
- Task heterogeneity: clients may optimize different losses and output spaces according to $\tau(c)$, yielding dissimilar gradients across tasks.

Moreover, real-time environments have client availability and update frequency asynchronous, leading to system heterogeneity and stale updates [2], [5], [6]. Thus, the optimization process must account for partial participation and non-uniform local progress.

C. Optimization Objective for Federated Multi-Task Learning

Let $\ell_{\tau(c)}(\theta_e, \theta_{\tau(c)}; \xi)$ be the task-specific loss cost for sample $\xi \in \mathcal{D}_c^t$. The global objective at round t is:

$$\min_{\theta_e, \{\theta_k\}} F_t(\Theta) = \sum_{c=1}^{C} p_c^t \, \mathbb{E}_{\xi \sim \mathcal{D}_c^t} \left[\ell_{\tau(c)}(\theta_e, \theta_{\tau(c)}; \xi) \right] + \lambda \, \Omega(\{\theta_k\}), \tag{1}$$

where p_c^t represents the weight of the aggregation (e.g., the weight per unit local sample), Ω is a regularizer to promote a stable cross-task learning and $\lambda \geq 0$ controls

the sharing-specialization trade-off. After localized updates, the server aggregates common encoder parameters by weighted averaging,

$$\theta_e^{t+1} = \sum_{c \in \mathcal{C}_t} \alpha_c^t \, \theta_{e,c}^{t+1}. \tag{2}$$

and updates custom task-specific parameters, as a result of task-wise aggregation,

$$\theta_k^{t+1} = \sum_{c \in \mathcal{C}_t \colon \tau(c)=k} \beta_c^t \, \theta_{k,c}^{t+1}. \tag{3}$$

In $\mathcal{C}_t$, the client subset participate at round t and α_c^t, β_c^t are normalized aggregation coefficients [2], [3], [7]. With privacy and heterogeneity requirements in place, the framework captures RT-FedMTL in real time by jointly optimizing both shared representations and task-specific parameters.

IV. PROPOSED METHOD

The proposed RT-FedMTL addresses the dual challenges of privacy-preserving NLU and resource-constrained edge computing. Our architecture integrates three core pillars: (1) a hierarchical federated architecture optimized for asynchronous real-time synchronization via WebSockets, (2) a multi-task learning strategy that leverages shared linguistic representations while maintaining task-specific specialization, and (3) the deployment of parameter-efficient SLMs to minimize the computational and communication footprint on edge devices.

A. System Overview

In this work, we propose a multistructure federated multi-task framework designed for real-time NLU applications. As illustrated in Fig. 1, a high level orchestration server, along with multiple, heterogeneous client nodes, structure the system. A parameter server is responsible for maintaining the global state at the center and controls the model aggregation logic. The communication relies on low-latency WebSockets for a bidirectional exchange of model updates and aggregated parameters.

Federated components and privacy constraints of a system are based on:

- Global server $\mathcal{S}$: coordinates client sampling, model broadcast, and aggregation.
- Distributed clients $\mathcal{C}$: edge devices/silos to store the raw text.
- Task set $\mathcal{K}$: the heterogeneous NLU tasks (intent classification, sentiment analysis, sequence labeling).
- Local datasets $\mathcal{D}_c^t$: single private, time-stamped data for client c at round t that is private.

Samples are never uploaded, for privacy purposes. Each client transmits only model updates (or compressed adapter and head updates) and serves as host-side aggregation [2], [8]. This is consistent with the usual federated privacy model in which data locality is preserved and collaborative training is promoted in the field.

B. Small Language Model Backbone

This framework is adapted with specialized SLMs to trade-off computational efficiency and language fluency. Our current solution also supports DistilBERT [13], BERT-Medium [20], MiniLM [14] architectures. These are shared feature extractors (encoder) that are fine-tuned collaboratively across the federated network. Our SLMs drastically minimize the communication overhead in contrast to standard-sized transformers.

C. Multi-Task Learning Strategies

We use a hybrid strategy of parameter-sharing. The lower layers of the transformer (encoder) are the same across the tasks that represent linguistic representation universally. In contrast, the top layers are task-specific classification or regression heads. We thus do an adaptive loss weighting mechanism to reduce the negative transfer:

$$\mathcal{L}_{total} = \sum_{j=1}^{T} \alpha_j \mathcal{L}_j \tag{4}$$

where α_j is dynamically tuned according to the convergence rate of each task j.

We define the parameters

$$\Theta = \{\theta_e, \{\theta_k\}_{k=1}^{K}\}, \tag{5}$$

for which θ_e is a common small-language-model encoder and θ_k is the task-based prediction head of task k [6], [7], [8]. Prediction is for an input-output pair (x, y) given task k.

$$\hat{y} = h_k\big(f_e(x; \theta_e); \theta_k\big), \tag{6}$$

where $f_e(\cdot)$ represents the shared encoder and $h_k(\cdot)$ refers to the task head. It allows for parameter sharing amongst transferable linguistic traits but maintains task-level specialization.

D. Federated Optimization and Real-Time Adaptation

The process of federated optimization is detailed in Algorithm 1. The system facilitates real-time adaptation by allowing clients to synchronize updates asynchronously via WebSockets. The server aggregates updates using a task-aware variant of FedAvg [2] and regularizes noise from heterogeneous sources (non-IID). Recent advancements in adaptive federated optimization [21] suggest that incorporating momentum and adaptive learning rates can further stabilize this process.

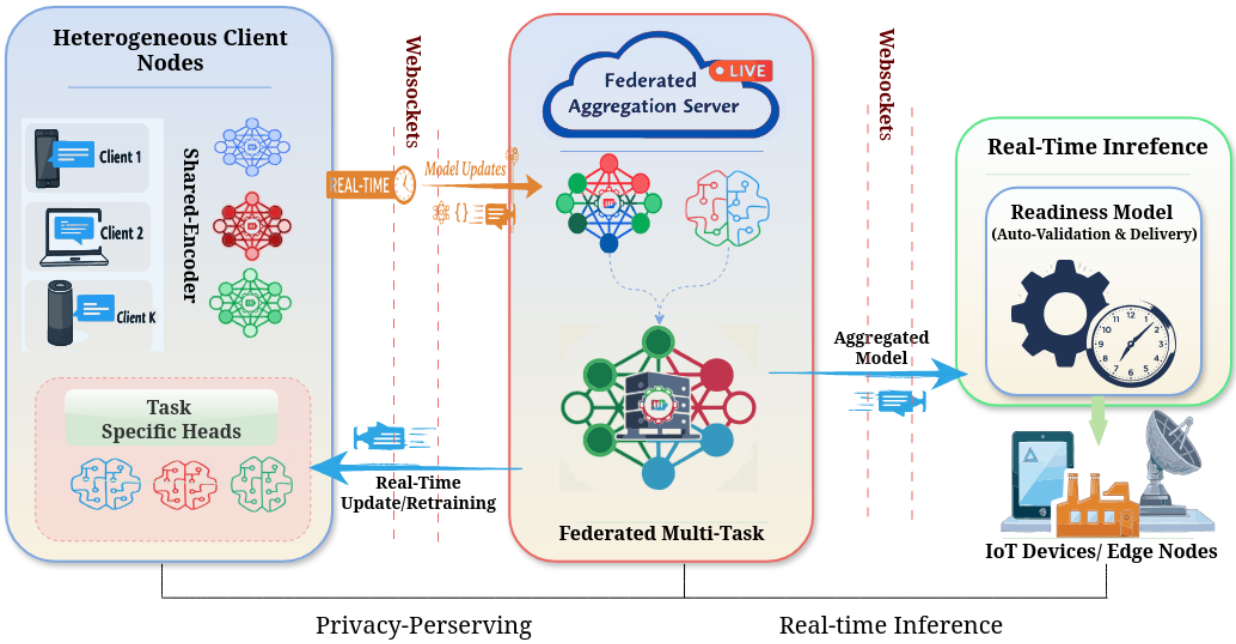

Fig. 1: Architecture of the Proposed Real-time Federated Multi-Task Learning Framework. The system enables seamless integration of streaming data processing and collaborative model refinement.

Algorithm 1 RT-FedMTL: Real-Time Federated Multi-Task Learning

Server Initialize: Shared core ϕ_0, Task heads $\{\psi_{j,0}\}_{j=1}^{T}$
for each round $t = 1, 2, \ldots$ **do**
  Server transmits $\phi_t, \psi_{Task(k),t}$ to client $k \in K$ via WebSockets
  for each client $k \in K$ in parallel **do**
    Local training: $\{\Delta\phi_k, \Delta\psi_{Task(k),k}\} \leftarrow \text{LocalUpdate}(\mathcal{D}_k, \phi_t, \psi_{Task(k),t})$
    Updates $\{\Delta\phi_k, \Delta\psi_{Task(k),k}\}$ sent to Server using WebSockets
  end for
  Task-Aware Aggregation:
    $\phi_{t+1} \leftarrow \phi_t + \eta \sum_{k=1}^{K} \frac{n_k}{N} \Delta\phi_k$ {Universal Knowledge}
  for each task $j \in \{1, \ldots, T\}$ in parallel **do**
    $\psi_{j,t+1} \leftarrow \psi_{j,t} + \eta \sum_{k \in K_j} \frac{n_k}{N_j} \Delta\psi_{j,k}$ {Heads Specialization}
  end for
  Server redistributes $\{\phi_{t+1}, \psi_{Task(k),t+1}\}$ for Local Validation
end for

E. Communication-Efficient Fine-Tuning (LoRA)

To optimize resource utilization on edge devices even further, we implement LoRA [22]. This includes when to update in local B, where instead of updating the full weight matrices $W \in \mathbb{R}^{d \times d}$, we optimize the low rank decomposition matrices A and B, where:

$$W' = W + \Delta W = W + BA. \tag{7}$$

$A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ with rank $r \ll d$. For big updates, only the matrices A and B are sent, minimizing communication volume to up to 99%.

V. EXPERIMENTS

This section evaluates the proposed RT-FedMTL framework across five thematic dimensions: overall baseline comparisons, the impact of MTL, paradigm performance gaps, real-time suitability, and communication efficiency.

A. Experimental Setup

1) Datasets and Tasks: The evaluation employs three core NLU benchmarks representing diverse NLU challenges from the GLUE benchmark [23].

SST-2 [24] is a binary classification task for sentiment polarity detection, comprising 66,477 training, 872 validation, and 1,821 test samples. Performance is measured by accuracy and F1-score [25].

QQP [26] is a binary classification task for paraphrase detection, with 323,415 training, 40,431 validation, and 39,081 test samples. Evaluation uses accuracy and F1-score [25].

STS-B [27] is a regression task for semantic similarity scoring over sentence pairs on a 1–5 scale, with 4,249

TABLE I: Experimental Design Matrix

Exp.	Paradigm	Task Config.	Clients	Data Dist.
1	Centralized MTL	Multi-Task	3	Full access
2	Centralized Single	SST-2 only	1	Full access
3	Centralized Single	QQP only	1	Full access
4	Centralized Single	STS-B only	1	Full access
5	Federated MTL	Multi-Task	3	IID
6	Federated MTL	Multi-Task	3	IID + LoRA
7	Federated MTL	Multi-Task	9	Non-IID (3×3)
8	Federated Single	QQP only	3	Non-IID
9	Federated Single	SST-2 only	3	Non-IID
10	Federated Single	STS-B only	3	Non-IID

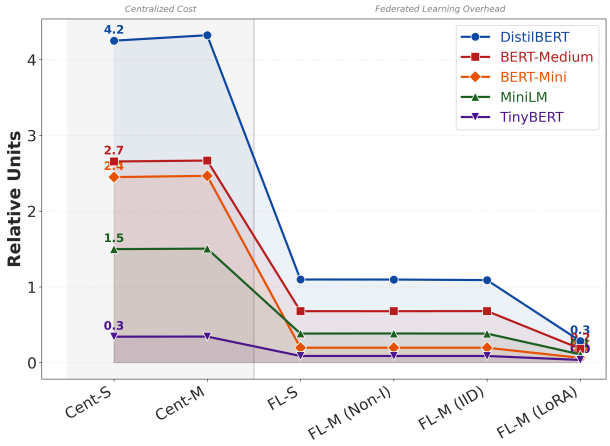

Fig. 2: Resource Usage Trajectory. Federated settings produce a clear 3-4× reduction in per-client resource usage relative to centralized training because each client holds only a data partition. LoRA further halves resource demands.

training, 1,500 validation, and 1,379 test samples. Performance is measured by Pearson [28] and Spearman correlation [29].

2) Federated Data Partitioning: IID Partitioning distributes data uniformly across clients (Experiments 5-6 in Table I, 3 clients, multi-task scenarios).

Non-IID Partitioning uses a Dirichlet distribution (α=0.5) to simulate realistic data heterogeneity, with 3 clients per task (9 total), yielding 107,805 / 22,159 / 1,416 samples per client for QQP, SST-2, and STS-B respectively (Experiments 7-10 in Table I).

Validation sets are *not* split across clients in any paradigm, ensuring consistent global model assessment.

3) Baselines: Three baselines are evaluated: (1) Centralized MTL: all 5 SLM variants trained on centralized data with shared encoder and task-specific heads, serving as the upper-bound reference (Exp. 1 in Table I); (2) Federated Single-Task: per-task federated experiments with 3 clients and non-IID data (Exps. 8-10 in Table I); and (3) Large-model federated approaches: comparison against larger LMs to assess communication efficiency

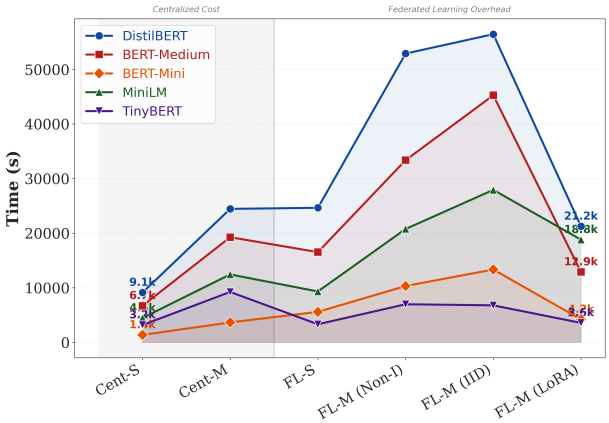

Fig. 3: Training Time Trajectory capturing the wall-clock training durations. The transition from centralized to federated multi-task settings introduces a significant overhead, which LoRA substantially offsets by limiting parameter transmission.

and performance trade-offs.

4) Evaluation Metrics: Task-level metrics are accuracy & F1-score [25] (SST-2, QQP) and Pearson [28] / Spearman correlation [29] (STS-B) [23]. System-level federated metrics include communication cost per round, convergence speed, and total training time.

5) Implementation Details: All experiments use learning rate 2×10^{-5} with 1,000-step warmup, batch size 8 for all experiments (Centralized to FL/FL-MTL), weight decay 0.01, gradient clipping (max-norm 1.0), and random seed 42. Centralized runs train for 25 epochs; federated clients perform 1 local epoch per round. Communication uses ports 8771-8775 with 60 s client timeout and 3,400 s round timeout, plus a 3-retry exponential-backoff mechanism.

B. Results and Discussion

1) Overall Performance (Comparison across Baselines): The results in Table IV demonstrate that small language models (SLMs) within the RT-FedMTL framework maintain high NLU performance. Centralized MTL serves as the benchmark, yet DistilBERT under federated MTL (IID) surprisingly achieves 0.9461 accuracy on SST-2, outperforming its centralized counterpart. MiniLM exhibits an exceptional resource-to-performance ratio, achieving 0.8966 and 0.8384 in QQP and STS-B tasks under IID settings, respectively—scores that match or exceed those of much larger models while requiring only 0.38 GB of GPU memory (compared to 1.09 GB for DistilBERT). This validates that in distributed NLU, model size is not the sole determinant of quality, as compact architectures like MiniLM can hold their own via federated optimization.

TABLE II: Resource Usage Trajectory (GB, per-client average)

Model	Cent-S	Cent-M	FL-S	FL-M (Non-IID)	FL-M (IID)	FL-M (LoRA)
DistilBERT	4.2492	4.3214	1.0972	1.0968	1.0896	0.2852
BERT-Medium	2.6549	2.6671	0.6795	0.6790	0.6807	0.1840
BERT-Mini	2.4487	2.4651	0.1969	0.1969	0.1969	0.0630
MiniLM	1.4982	1.5045	0.3839	0.3846	0.3835	0.1090
TinyBERT	0.3427	0.3440	0.0881	0.0881	0.0881	0.0356

Cent-S: Centralized Single-Task, Cent-M: Centralized Multi-Task, FL-S: Federated Single-Task, FL-M: Federated Multi-Task, IID/Non-IID: Data Distribution, LoRA: Low-Rank Adaptation.

TABLE III: Training Time Trajectory (seconds)

Model	Cent-S	Cent-M	FL-S	FL-M (Non-IID)	FL-M (IID)	FL-M (LoRA)
DistilBERT	9,122	24,450	24,648	52,904	56,454	21,245
BERT-Medium	6,693	19,257	16,527	33,375	45,258	12,886
BERT-Mini	1,359	3,653	5,581	10,318	13,341	4,192
MiniLM	4,473	12,437	9,317	20,766	27,930	18,808
TinyBERT	3,170	9,228	3,308	6,975	6,778	3,561

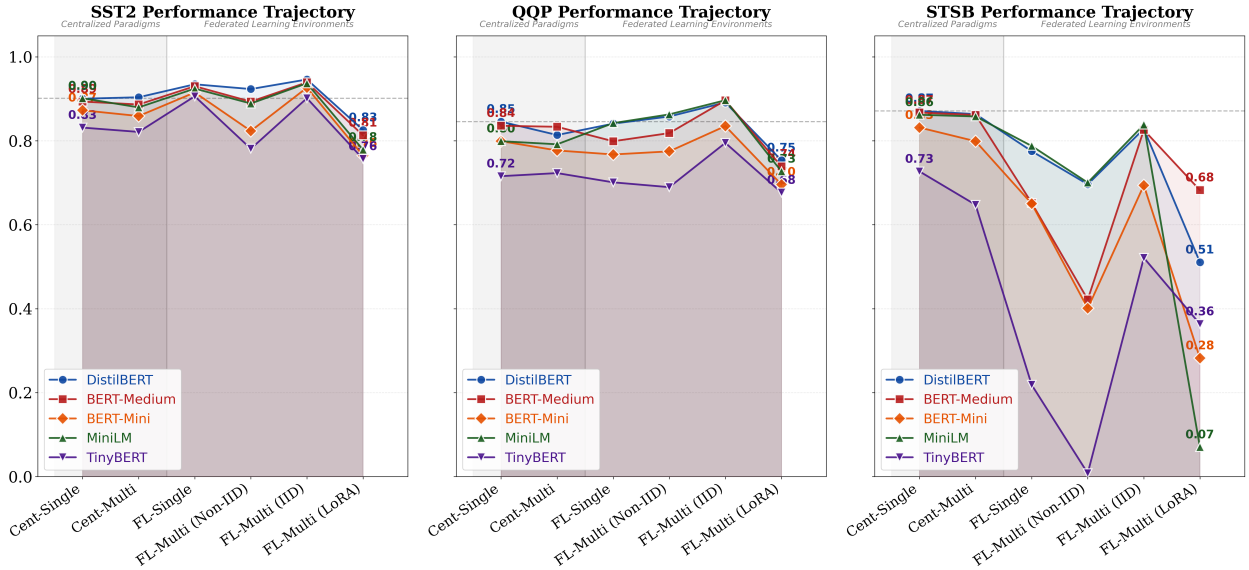

Fig. 4: Performance across six paradigm-complexity stages for five SLM architectures. Dashed lines separate centralized and federated settings.

2) Effect of Multi-Task Learning: MTL provides significant advantages in federated settings by leveraging shared representations across related tasks. As shown in Figure 4 and Table IV, comparing FL-Single and FL-MTL (Non-IID) reveals complex task interactions. While SST-2 suffers modest negative transfer in certain configurations, regression tasks like STS-B often suffer more pronounced drops under Non-IID conditions. However, classification tasks generally demonstrate better positive transfer; for instance, QQP benefits from the features learned via SST-2. Larger SLMs like DistilBERT and BERT-Medium show higher resilience to negative transfer due to their greater representational capacity.

3) Federated Learning Performance: The transition from centralized to federated learning introduces a "Reliability Gap," characterized by a moderate degradation ($\leq 5\%$) in classification accuracy for tasks like SST-2 and QQP. This gap is most visible in the STS-B Pearson correlation, which drops significantly for the smallest architectures like TinyBERT under non-IID MTL conditions. Figure 4 maps this trajectory, showing how the proposed RT-FedMTL framework manages to close this gap by stabilizing performance across tasks. DistilBERT consistently proves to be the most robust architecture for sentiment-heavy tasks, exhibiting minimal performance decay even under extreme non-IID skew (0.9232 Acc. on SST-2).

TABLE IV: Comparative Analysis of SLM Performance across Paradigms

Model	Paradigm	Task	Dist.	Tuning	SST-2		QQP		STS-B		System	System
					Acc.	F1	Acc.	F1	Pear.	Spea.	Time (s)	Res (GB)
DistilBERT [13]	FL	ST	Non-IID	FFT	—	—	0.8408	0.7724	—	—	34904.44	1.0956
	FL	ST	Non-IID	FFT	—	—	—	—	0.7753	0.7421	15452.52	1.0988
	FL	ST	Non-IID	FFT	0.9346	0.9392	—	—	—	—	23586.44	1.0972
	FL	MTL (ours)	IID	LoRA	0.8257	0.8433	0.7535	0.6754	0.5108	0.4919	21245.26	0.2852
	FL	MTL (ours)	Non-IID	FFT	0.9232	0.9298	0.8579	0.8054	0.6963	0.6967	52903.97	1.0968
	FL	MTL (ours)	IID	FFT	0.9461	0.9514	0.8913	0.8610	0.8284	0.8136	56454.46	1.0896
	Cent.	ST	IID	FFT	—	—	—	—	0.8712	0.8691	4561.78	4.2470
	Cent.	ST	IID	FFT	0.9002	0.9061	—	—	—	—	18514.82	4.2460
	Cent.	ST	IID	FFT	—	—	0.8456	0.8934	—	—	4290.59	4.2547
	Cent.	MTL	IID	FFT	0.9037	0.9077	0.8137	0.8778	0.8635	0.8614	24449.68	4.3214
BERT-Medium [20]	FL	ST	Non-IID	FFT	—	—	0.7989	0.6852	—	—	23880.89	0.6789
	FL	ST	Non-IID	FFT	—	—	—	—	0.6521	0.6212	10261.92	0.6810
	FL	ST	Non-IID	FFT	0.9300	0.9348	—	—	—	—	15439.04	0.6786
	FL	MTL (ours)	Non-IID	FFT	0.8933	0.8988	0.8183	0.7441	0.4219	0.4132	33374.50	0.6790
	FL	MTL (ours)	IID	FFT	0.9392	0.9450	0.8962	0.8655	0.8258	0.8052	45258.21	0.6807
	FL	MTL (ours)	IID	LoRA	0.8131	0.8362	0.7388	0.6805	0.6823	0.6512	12885.74	0.1840
	Cent.	ST	IID	FFT	—	—	—	—	0.8673	0.8648	3381.83	2.6467
	Cent.	ST	IID	FFT	0.8933	0.8970	—	—	—	—	13492.62	2.6711
	Cent.	ST	IID	FFT	—	—	0.8358	0.8859	—	—	3205.51	2.6467
	Cent.	MTL	IID	FFT	0.8865	0.8884	0.8333	0.8777	0.8615	0.8584	19257.40	2.6671
BERT-Mini [20]	FL	ST	Non-IID	FFT	—	—	0.7674	0.6201	—	—	9642.59	0.1969
	FL	ST	Non-IID	FFT	—	—	—	—	0.6504	0.6150	2547.10	0.1969
	FL	ST	Non-IID	FFT	0.9151	0.9196	—	—	—	—	4552.75	0.1969
	FL	MTL (ours)	Non-IID	FFT	0.8234	0.8226	0.7748	0.6668	0.4009	0.3939	10317.55	0.1969
	FL	MTL (ours)	IID	LoRA	0.7649	0.7915	0.6958	0.5602	0.2824	0.2773	4191.67	0.0630
	FL	MTL (ours)	IID	FFT	0.9266	0.9329	0.8350	0.7981	0.6940	0.6550	13341.01	0.1969
	Cent.	ST	IID	FFT	—	—	—	—	0.8316	0.8283	411.45	2.4487
	Cent.	ST	IID	FFT	0.8727	0.8773	—	—	—	—	3348.26	2.4487
	Cent.	ST	IID	FFT	—	—	0.7990	0.8536	—	—	318.61	2.4487
	Cent.	MTL	IID	FFT	0.8589	0.8650	0.7770	0.8580	0.7989	0.7973	3652.50	2.4651
MiniLM [14]	FL	ST	Non-IID	FFT	—	—	0.8421	0.7644	—	—	14583.05	0.3842
	FL	ST	Non-IID	FFT	—	—	—	—	0.7877	0.7489	4920.48	0.3840
	FL	ST	Non-IID	FFT	0.9243	0.9293	—	—	—	—	8447.20	0.3836
	FL	MTL (ours)	IID	FFT	0.9369	0.9428	0.8966	0.8662	0.8384	0.8197	27930.06	0.3835
	FL	MTL (ours)	Non-IID	FFT	0.8888	0.8974	0.8629	0.8150	0.7003	0.7061	20765.57	0.3846
	FL	MTL (ours)	IID	LoRA	0.7787	0.8102	0.7271	0.6060	0.0703	0.0621	18808.08	0.1090
	Cent.	ST	IID	FFT	—	—	—	—	0.8620	0.8592	2696.27	1.4929
	Cent.	ST	IID	FFT	0.9014	0.9051	—	—	—	—	8334.10	1.4950
	Cent.	ST	IID	FFT	—	—	0.7990	0.8656	—	—	2388.13	1.5067
	Cent.	MTL	IID	FFT	0.8796	0.8829	0.7917	0.8649	0.8580	0.8560	12437.12	1.5045
TinyBERT [30]	FL	ST	Non-IID	FFT	—	—	0.7010	0.3887	—	—	6226.63	0.0881
	FL	ST	Non-IID	FFT	—	—	—	—	0.2186	0.1755	1182.03	0.0881
	FL	ST	Non-IID	FFT	0.9060	0.9122	—	—	—	—	2514.06	0.0881
	FL	MTL (ours)	IID	FFT	0.9014	0.9145	0.7950	0.7494	0.5212	0.4754	6778.00	0.0881
	FL	MTL (ours)	Non-IID	FFT	0.7810	0.7782	0.6895	0.4733	0.0081	0.0063	6975.28	0.0881
	FL	MTL (ours)	IID	LoRA	0.7569	0.7902	0.6766	0.4475	0.3636	0.3505	3560.68	0.0356
	Cent.	ST	IID	FFT	—	—	—	—	0.7270	0.7230	2337.42	0.3427
	Cent.	ST	IID	FFT	0.8314	0.8358	—	—	—	—	4857.08	0.3427
	Cent.	ST	IID	FFT	—	—	0.7157	0.8139	—	—	2315.39	0.3427
	Cent.	MTL	IID	FFT	0.8211	0.8263	0.7230	0.8101	0.6474	0.6426	9227.55	0.3440

ST = Single-Task, MTL = Multi-Task Learning, Cent. = Centralized, LoRA = Low-Rank Adaptation, FFT = Full Fine-Tuning, Pear. = Pearson Correlation, Spea. = Spearman Correlation.

Quantitatively, the IID federated results represent the framework's optimal performance, showing that MTL can yield better results than isolated centralized training for specific models (e.g., DistilBERT reaching 0.9461 on SST-2). The non-IID metrics highlight the framework's resilience under data heterogeneity; while classification tasks like SST-2 maintain stable performance (e.g., 0.9232 for DistilBERT), regression tasks like STS-B exhibit greater sensitivity. MiniLM consistently emerges as the strongest versatile option across both IID and non-IID distributions, winning 2 out of 3 benchmark tasks (QQP and STS-B) with a nearly 3× reduction in memory footprint compared to DistilBERT. This demonstrates that while the framework is resilient for categorical benchmarks, model selection should be tailored: DistilBERT for sentiment robustness and MiniLM for resource-constrained multi-task stability.

4) Real-Time Inference Analysis: For deployment on edge devices where real-time inference is critical, resource consumption and model footprint are the defining metrics. Figure 2 and Table II highlight that FL reduces individual client GPU memory demands by 77-93%

relative to centralized training. TinyBERT and BERT-Mini emerge as the leading candidates for extremely resource-constrained devices (< 0.1 GB and < 0.2 GB per client respectively), albeit with bounded performance on complex tasks. MiniLM and DistilBERT provide a superior performance-efficiency balance for most real-time scenarios, maintaining high accuracy while keeping per-client resource usage significantly lower than the centralized baseline.

5) Communication Efficiency: Communication overhead is the primary bottleneck in federated MTL, with wall-clock training times increasing by 2.2-3.7× compared to centralized training (Table III). However, as illustrated in Figure 3, the integration of PEFT via LoRA [22] substantially mitigates this cost. LoRA reduces federated training time by 38-62% for most models by transmitting only low-rank adapter weights instead of full model parameters. This optimization enables the framework to maintain competitive accuracy while drastically reducing the bandwidth requirements, making it suitable for bandwidth-limited edge deployments.

VI. CONCLUSION

In this work, we presented a comprehensive investigation into RT-FedMTL using SLMs across diverse NLU benchmarks. Our systematic evaluation across five architectures provided critical insights for the deployment of decentralized NLU systems. Our results demonstrate that FL effectively preserves data privacy while significantly optimizing resource distribution, achieving up to 93% reduction in per-client GPU resource consumption. We identified a critical performance-efficiency trade-off: DistilBERT consistently offered the most robust performance for sentiment analysis, showing superior resilience to negative transfer in SST-2. However, for a unified multi-task suite, MiniLM emerged as the most versatile and efficient architecture, outperforming larger models in QQP and STS-B tasks despite being approximately 3× smaller in memory footprint (0.38 GB vs 1.09 GB). For scenarios where memory and bandwidth are the primary bottlenecks, architectures like MiniLM provide the optimal balance of stability and throughput for real-time federated multi-task environments.

Future research will focus on developing adaptive task-sharing strategies to further mitigate negative transfer in multi-head architectures. Additionally, we plan to explore personalized federated learning techniques and the integration of differentially private mechanisms within the real-time communication layer, ensuring that efficiency gains do not compromise privacy in production NLU deployments.

REFERENCES

[1] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine*, 37(3):50–60, 2020.

[2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. Communication-efficient learning of deep networks from decentralized data. In *AISTATS*, 2017.

[3] V. Smith, C.-K. Chiang, M. Sanjabi, and A. Talwalkar. Federated multi-task learning. In *NeurIPS*, 2017.

[4] B. Lin, C. He, Z. An, et al. FedNLP: A benchmarking framework for federated learning on natural language processing. In *NAACL*, 2021.

[5] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith. Federated optimization in heterogeneous networks. In *MLSys*, 2020.

[6] Y. Chen, T. Zhang, X. Jiang, Q. Chen, C. Gao, and W. Huang. FedBone: Towards large-scale federated multi-task learning. *arXiv preprint arXiv:2306.17465*, 2024.

[7] J. Zhou, W. Bao, J. Wang, D. Zhang, X. Zhang, and Y. Zhang. Multi-task federated learning with encoder–decoder structure. *arXiv preprint arXiv:2504.09800*, 2025.

[8] A. Elbakary, C. Ben Issaid, T. ElBatt, K. Seddik, and M. Bennis. MIRA: A method of federated multi-task learning for large language models. *arXiv preprint*, 2025.

[9] S. P. Karimireddy, S. Kale, M. Mohri, et al. SCAFFOLD: Stochastic controlled averaging for federated learning. In *ICML*, 2020.

[10] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn. Gradient surgery for multi-task learning. *arXiv preprint arXiv:2001.06782*, 2020.

[11] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al. Parameter-efficient transfer learning for NLP. In *ICML*, 2019.

[12] J. Pfeiffer, A. Rücklé, C. Poth, et al. AdapterHub: A framework for adapting transformers. In *EMNLP-System Demonstrations*, 2020.

[13] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[14] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *NeurIPS*, 2020.

[15] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut. ALBERT: A Lite BERT for self-supervised learning of language representations. In *ICLR*, 2020.

[16] S. Wang, et al. EdgeDistilBERT: A lightweight multi-task federated learning framework for resource-constrained devices. *arXiv preprint*, 2025.

[17] X. Li, M. Jiang, X. Zhang, et al. FedBN: Federated learning on non-IID data with batch normalization. In *ICLR*, 2021.

[18] Z. Huo, M. Bin Ahmad, S. Guu, et al. FedPara: Low-rank Hadamard product for communication-efficient federated learning. In *ICLR*, 2022.

[19] Y. Khan, D. Sánchez, and J. Domingo-Ferrer. Federated learning-based natural language processing: A systematic literature review. *Future Generation Computer Systems*, 2024.

[20] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.

[21] S. Reddi, Z. Charles, M. Zaheer, et al. Adaptive federated optimization. In *ICLR*, 2021.

[22] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

[23] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *ICLR*, 2019.

[24] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Ng, and C. Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *EMNLP*, 2013.

[25] C. J. Van Rijsbergen. *Information Retrieval*. Butterworth-Heinemann, 2nd edition, 1979.

[26] L. Sharma, L. Graesser, N. Nangia, and U. Evci. Natural language understanding with the quora question pairs dataset. *arXiv preprint arXiv:1907.01041*, 2019.

[27] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia. SemEval-2017 Task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In *SemEval*, 2017.

[28] K. Pearson. VII. Note on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London*, 58:240–242, 1895.

[29] C. Spearman. The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101, 1904.

[30] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu. TinyBERT: Distilling BERT for natural language understanding. In *Findings of EMNLP*, 2020.